

AirOne — Quick Start Guide

Get the AirOne Ground Station running in **under 5 minutes**.

Prerequisites

- Python 3.10, 3.11, or 3.12
 - 2 GB RAM (4 GB recommended for ML features)
 - 500 MB disk space
-

Installation

1. Navigate to the application directory

```
cd airone
```

2. Install Python dependencies

```
pip install -r requirements.txt
```

3. (Optional) Install ML packages for full detector coverage

```
pip install scikit-learn torch
```

Configuration

Generate a secure JWT secret

```
export AIRONE_JWT_SECRET="$(python3 -c 'import secrets; print(secrets.token_urlsafe(48))')"
```

Or set it manually (minimum 32 characters)

```
export AIRONE_JWT_SECRET="your_very_secure_secret_at_least_32_characters_long"
```

Running the System

Option 1: Live Mode (with real CanSat hardware)

Connect the **ground LoRa E22** module (transparent mode, M0=M1=GND) to your laptop via a USB-serial bridge, then:

Auto-discover the radio:

```
python3 launcher.py
```

Or specify the port/baud explicitly (recommended for E22 transparent mode):

```
python3 launcher.py --serial-port /dev/ttyUSB0 --baud 115200 --force-baud
```

What happens: - System starts and binds API to `http://localhost:5000` - Opens the E22 serial link (auto-discover, `AIRONE_SERIAL_PORT`, or `--serial-port`) and streams live telemetry through the real pipeline - Prints an explicit link state: `LIVE (...)`, or `NOT_CONFIGURED (...)` when no radio is found — **no data is ever fabricated** - Logs to `logs/airone.jsonl`

Flight firmware: the matching ESP32 sketch lives in `firmware/airone_cansat/`. It emits the exact AirOne binary frames (verified byte-for-byte against the ground station). Full wiring and setup: `docs/hardware_connection.md`.

Option 2: Simulation Mode (no hardware needed)

`python3 launcher.py --simulate`

What happens: - Generates synthetic CanSat telemetry (ISA atmosphere model) - Sends through full binary protocol + CRC - Processes through 20-stage pipeline - Runs all 37 scientific analyses - Perfect for testing, demos, and development

Option 3: With GUI (requires PyQt5)

`python3 launcher.py --gui --simulate`

What happens: - Launches six-tab PyQt5 ground station interface - Mission Control, Science Dashboard, GNSS Map, Replay, AI Diagnostics, Config/System - Auto-refreshes with live telemetry

Verify Installation

Run the full test suite:

`pytest -v`

Expected output: 154 passed in ~25s

Access the API

Once running, the API is available at `http://localhost:5000/api/v1/`

0. Create the first administrator (once)

No accounts ship with the software:

`AIRONE_ADMIN_PASSWORD='<strong password, 12+ chars>' python3 launcher.py --create-admin`
or: `python3 launcher.py --create-admin --generate-password` (printed once)

1. Login to get JWT token

```
curl -X POST http://localhost:5000/api/v1/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username": "admin", "password": "<the password you set>"}'
```

Response:

```
{  
  "success": true,  
  "data": {  
    "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2Vybm91dCI6ImFkbWlnIiwiaWF0IjoxNjU0OTU0ODUwLCJyb2wiOiJhZG9pbnQifQ",  
    "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2Vybm91dCI6ImFkbWlnIiwiaWF0IjoxNjU0OTU0ODUwLCJyb2wiOiJhZG9pbnQifQ",  
    "user": {  
      "username": "admin",  
      "role": "admin"  
    }  
  }  
}
```

2. Use the token for authenticated requests

Save token

```
TOKEN="eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2Vybm91dCI6ImFkbWlnIiwiaWF0IjoxNjU0OTU0ODUwLCJyb2wiOiJhZG9pbnQifQ"
```

Get system health

```
curl http://localhost:5000/api/v1/system/health \  
-H "Authorization: Bearer $TOKEN"
```

Get telemetry measurements

```
curl http://localhost:5000/api/v1/telemetry/measurements?limit=10 \  
-H "Authorization: Bearer $TOKEN"
```

List scientific analyses

```
curl http://localhost:5000/api/v1/scientific/analyses \  
-H "Authorization: Bearer $TOKEN"
```

Check ML detector availability

```
curl http://localhost:5000/api/v1/ml/detectors \  
-H "Authorization: Bearer $TOKEN"
```

Docker Deployment (Production)

1. Set production JWT secret

```
export AIRONE_JWT_SECRET="$(python3 -c 'import secrets;  
print(secrets.token_urlsafe(48))')"
```

*# 2. Start with Docker Compose (refuses to start without AIRONE_JWT_SECRET;
the API is published on 127.0.0.1:5000 only)*

```
docker compose up -d
```

```
# 2b. Create the administrator inside the container (once)
AIRONE_ADMIN_PASSWORD='<strong password>' docker compose exec \
-e AIRONE_ADMIN_PASSWORD airone python3 launcher.py --create-admin
```

```
# 3. Check logs
docker-compose logs -f airone
```

```
# 4. Access API
curl http://localhost:5000/api/v1/system/health
```

Accounts

There are no default credentials. Create accounts with the launcher:

```
python3 launcher.py --create-user ops1 --role OPERATOR --generate-password
python3 launcher.py --reset-password ops1      # uses
AIRONE_ADMIN_PASSWORD
python3 launcher.py --list-users
```

Roles: VIEWER < OPERATOR < SCIENTIST < ENGINEER < ADMIN. Admins can also manage accounts through POST /api/v1/users (see docs/api_reference.md).

For an isolated demo only, AIRONE_ALLOW_DEFAULT_USERS=1 seeds admin/AirOneAdmin!2026 and viewer/viewer123; both must change their password on first login (POST /api/v1/auth/change-password).

File Locations

- **Database:** data/airone.db
 - **Logs:** logs/airone.jsonl, logs/audit.jsonl
 - **Config:** config/default_config.yaml
 - **Documentation:** docs/ (13 comprehensive guides)
-

Troubleshooting

“InsecureConfigError: Refusing to start with default JWT secret”

Solution: Set AIRONE_JWT_SECRET environment variable (min 32 chars).

```
export AIRONE_JWT_SECRET="your_secure_secret_minimum_32_characters_long"
```

“No module named ‘PyQt5’”

Solution: Either install PyQt5 or run headless:

Option 1: Install PyQt5

```
pip install PyQt5
```

Option 2: Run headless (no GUI)

```
export QT_QPA_PLATFORM=offscreen
```

```
python3 launcher.py --simulate
```

“Serial port not found”

Solution: 1. Check device is connected: `ls /dev/ttyUSB*` (Linux) / Device Manager (Windows) 2. Add user to dialout group: `sudo usermod -a -G dialout $USER` (then re-login) 3. Specify the port explicitly: `python3 launcher.py --serial-port /dev/ttyUSB0` (or `export AIRONE_SERIAL_PORT=/dev/ttyUSB0`) 4. Port opens but no packets? Baud mismatch — match the E22 UART baud: `python3 launcher.py --serial-port /dev/ttyUSB0 --baud 115200 --force-baud` 5. Full wiring, E22 configuration, and troubleshooting table: [docs/hardware_connection.md](#)

The launcher never fabricates data: with no radio it reports NOT_CONFIGURED and streams nothing (use `--simulate` for isolated testing).

ML detectors show as “unavailable”

Solution: Install optional dependencies:

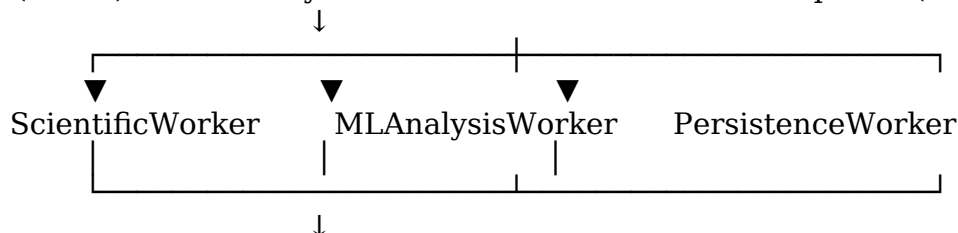
```
pip install scikit-learn torch
```

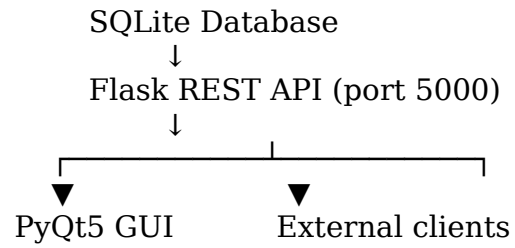
Next Steps

1. **Read the documentation:** [docs/operator_manual.md](#) for mission operations
 2. **Explore the API:** [docs/api_reference.md](#) for all endpoints
 3. **Understand the science:** [docs/scientific_methods.md](#) for all 37 analyses
 4. **Review security:** [docs/security_model.md](#) before production deployment
 5. **Deploy to production:** [docs/deployment.md](#) for Docker/systemd setup
-

Quick Architecture Overview

CanSat (Serial) → TelemetryReceiver → PacketProcessor → Pipeline (20 stages)





Support

For detailed troubleshooting, consult: - docs/troubleshooting.md — Common issues and solutions - docs/failure_modes.md — How each subsystem degrades - logs/airone.jsonl — Structured system logs

System is ready. Happy CanSat operations! 🚀